

AI Report

We are moderately confident this text is

AI Generated

AI Probability

73%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism



The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

CS 69/169: Lab 5 - Reverse Engineering Report - 2/1/2026

Wayner Barrios Quiroga

CS 69/169: Lab 5 - Reverse Engineering Report

Name: Wayner Barrios (way.gr@dartmouth.edu)

DartID: f005d5s

Date: January 30, 2026

Tools I Used

- objdump (GNU Binutils 2.42) - my go-to for disassembly

- xxd - hex dumps

- Python 3.12.2 - for the patching script

- file, strings, readelf, cmp - the usual suspects

My Setup

I did all this work on my server marvin (Ubuntu 24.04.3 LTS, kernel 6.8.0-90-generic).

It's running an AMD EPYC 7763 which is overkill for this but whatever.

x86-64, little-endian.

Quick Syscall Reference

I kept looking this up so here's the table I used.

On x86-64 Linux the syscall number goes in RAX, then you hit the `syscall` instruction:

Syscall	Number	RDI	RSI	RDX	Returns
---------	--------	-----	-----	-----	---------

-----	-----	-----	-----	-----	-----
-------	-------	-------	-------	-------	-------

sys_read	0	fd	buf	count	bytes read
----------	---	----	-----	-------	------------

sys_write	1	fd	buf	count	bytes written
-----------	---	----	-----	-------	---------------

sys_open	2	filename	flags	mode	fd
----------	---	----------	-------	------	----

sys_exit	60	status	-	-	doesn't return
----------	----	--------	---	---	----------------

Starting Out: What's This Binary?

So first thing, let's see what we're dealing with:

```
```bash
```

```
file bin/challenge.bin
```

```
```
```

```
```
```

```
bin/challenge.bin: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=c684c25fc79e15ee8b59ae9ad013f9f3dd79bc0e, for GNU/Linux 4.4.0, stripped
```

```
```
```

64-bit ELF, dynamically linked, and stripped.

Great, no symbols.

This means I gotta work with raw addresses which is annoying but fine.

Does It Even Run?

Let's just run it and see what happens:

```
```bash
```

```
./bin/challenge.bin
```

```
'''
```

```
'''
```

Segmentation fault (core dumped)

```
'''
```

Yep, crashes immediately.

Segfault means it's touching memory it shouldn't.

Now I need to figure out where and why.

## ## Looking for Clues

Before diving into disassembly, I always run `strings` to see if there's anything obvious:

```
```bash
```

```
strings bin/challenge.bin
```

```
'''
```

Interesting bits:

```
'''
```

mmap:

mprotect:

directory/newdirectory/file.txtA

```
'''
```

Okay so it's gonna mmap some memory, mprotect it (probably to make it executable), and there's a filepath.

That trailing 'A' is suspicious - probably gets XORed to become a null terminator ($0x41 \oplus 0x41 = 0x00$).

Classic shellcode trick to avoid null bytes.

Digging Into the Disassembly

Time to actually look at the code.

I found main at 0x1179:

```
```bash
```

```
objdump -d bin/challenge.bin | grep -A50 "1179:"
```

```
```
```

```
```
```

```
1179:55 push %rbp
```

```
117a:48 89 e5 mov %rsp,%rbp
```

```
117d:48 8d 05 dc 2e 00 00 lea 0x2edc(%rip),%rax # 4060
```

```
1184:be 67 00 00 00 mov $0x67,%esi
```

```
1189:48 89 c7 mov %rax,%rdi
```

```
118c:e8 02 00 00 00 call 1193
```

```
1191:5d pop %rbp
```

```
1192:c3 ret
```

```
```
```

So main loads address 0x4060 and size 0x67 (103 bytes), then calls whatever's at 0x1193. That's gotta be the shellcode executor.

```
## The Shellcode Executor
```

```
Looking at 0x1193:
```

```
```bash
```

```
objdump -d bin/challenge.bin | grep -A60 "1193:"
```

```
```
```

```
```
```

```
1193:55 push %rbp
```

```
1194:48 89 e5 mov %rsp,%rbp
```

```
1197:48 83 ec 30 sub $0x30,%rsp
```

```
...
```

```
11d4:e8 57 fe ff ff call 1030 <mmap@plt>
```

```
...
```

```
1207:e8 34 fe ff ff call 1040 <memcpy@plt>
```

```
...
```

```
1228:e8 33 fe ff ff call 1060 <mprotect@plt>
```

```
...
```

```
124f:ff d0 call *%rax
```

```
'''
```

Okay so the flow is:

1. mmap some memory (read+write)
2. memcpy the shellcode into it
3. mprotect to add execute permission
4. call into the shellcode

Pretty standard shellcode loader pattern.

The crash must be happening inside the shellcode itself.

### ## Getting the Shellcode Out

The shellcode lives in .data.

Let me dump it:

```
```bash
```

```
objdump -s -j.data bin/challenge.bin
```

'''

'''

Contents of section.data:

4060 eb3f5880 771f4148 31c00402 4831f60f.

?X.w.AH1...H1..

4070 056681ec ff0f488d 34244889 c74831d2.f....H.4\$H..H1.

4080 66baff0f 4831c00f 054831ff 4080c709 f...H1...H1.

@...

4090 4889c248 31c00401 0f054831 c0043c0f H..H1.....H1..<.

40a0 05e8bcff ffff6469 72656374 6f72792f.....directory/

40b0 6e657764 69726563 746f7279 2f66696c newdirectory/fil

40c0 652e7478 744100 e.txtA.

'''

I can see the path string at the end.

Now let me actually disassemble the shellcode itself:

```bash

dd if=bin/challenge.bin bs=1 skip=\$((0x3060)) count=\$((0x67)) of=/tmp/shellcode.bin 2>/dev/null

objdump -D -b binary -m i386:x86-64 /tmp/shellcode.bin

'''

'''

0:eb 3f jmp 0x41

2:58 pop %rax

3:80 77 1f 41 xorb \$0x41,0x1f(%rdi)

7:48 31 c0 xor %rax,%rax

a:04 02 add \$0x2,%al

c:48 31 f6 xor %rsi,%rsi

f:0f 05 syscall

11:66 81 ec ff 0f sub \$0xffff,%sp

16:48 8d 34 24 lea (%rsp),%rsi

1a:48 89 c7 mov %rax,%rdi

1d:48 31 d2 xor %rdx,%rdx

20:66 ba ff 0f mov \$0xffff,%dx

24:48 31 c0 xor %rax,%rax

27:0f 05 syscall

29:48 31 ff xor %rdi,%rdi

2c:40 80 c7 09 add \$0x9,%dil

30:48 89 c2 mov %rax,%rdx

33:48 31 c0 xor %rax,%rax

36:04 01 add \$0x1,%al

38:0f 05 syscall

3a:48 31 c0 xor %rax,%rax

3d:04 3c add \$0x3c,%al

3f:0f 05 syscall

41:e8 bc ff ff ff call 0x2

46:"directory/newdirectory/file.txtA"

...

Alright NOW we're getting somewhere.

### ## What's the Shellcode Doing?

This is using the classic jmp-call-pop trick to get the string address.  
Here's how it works:

1. `jmp 0x41` - skip ahead to the call
2. `call 0x2` - this pushes the return address (which points to the string!) and jumps back
3. `pop` grabs that address off the stack

Then it does a bunch of syscalls: open the file, read it, write to fd 9, and exit.

### ## Found It!

Look at this:

```
```asm
```

```
2:58 pop %rax; string address → RAX
```

```
3:80 77 1f 41 xorb $0x41,0x1f(%rdi); wait... RDI?  
?
```

```
```
```

There's the bug!

The `pop` puts the string address into RAX, but then the next instruction tries to XOR using RDI as a base.  
But RDI was never set to anything!

It's just garbage from before, so when we try to access memory at `RDI + 0x1f`, we segfault.

Also sys\_open expects the filename pointer in RDI, not RAX.  
So even if we got past the XOR, the open would fail.

The fix is dead simple: change `pop %rax` (0x58) to `pop %rdi` (0x5F).  
One byte.

### ## Register Trace (for my own sanity)

**\*\*Buggy version:\*\***

| Instruction | RAX | RDI | What happens |
|-------------|-----|-----|--------------|
|-------------|-----|-----|--------------|

|       |      |      |       |
|-------|------|------|-------|
| ----- | ---- | ---- | ----- |
|-------|------|------|-------|

|            |   |  |  |
|------------|---|--|--|
| `jmp 0x41` | ? |  |  |
|------------|---|--|--|

|         |            |  |  |
|---------|------------|--|--|
| garbage | jump ahead |  |  |
|---------|------------|--|--|

|            |   |  |  |
|------------|---|--|--|
| `call 0x2` | ? |  |  |
|------------|---|--|--|

|         |                  |  |  |
|---------|------------------|--|--|
| garbage | push string addr |  |  |
|---------|------------------|--|--|

|            |      |         |                      |
|------------|------|---------|----------------------|
| `pop %rax` | &str | garbage | BUG: wrong register! |
|------------|------|---------|----------------------|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|                          |      |         |                 |
|--------------------------|------|---------|-----------------|
| `xorb \$0x41,0x1f(%rdi)` | &str | garbage | BOOM - segfault |
|--------------------------|------|---------|-----------------|

**\*\*After fix:\*\***

| Instruction | RAX | RDI | What happens |
|-------------|-----|-----|--------------|
|-------------|-----|-----|--------------|

|       |      |      |       |
|-------|------|------|-------|
| ----- | ---- | ---- | ----- |
|-------|------|------|-------|

|            |   |  |  |
|------------|---|--|--|
| `pop %rdi` | ? |  |  |
|------------|---|--|--|

|      |                            |  |  |
|------|----------------------------|--|--|
| &str | string addr in right place |  |  |
|------|----------------------------|--|--|

|        |   |  |  |
|--------|---|--|--|
| `xorb` | ? |  |  |
|--------|---|--|--|

|      |                            |  |  |
|------|----------------------------|--|--|
| &str | null-terminates the string |  |  |
|------|----------------------------|--|--|

|                |   |      |        |
|----------------|---|------|--------|
| syscall (open) | 2 | &str | works! |
|----------------|---|------|--------|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|                |   |    |            |
|----------------|---|----|------------|
| syscall (read) | 0 | fd | reads file |
|----------------|---|----|------------|

|                 |   |   |                 |
|-----------------|---|---|-----------------|
| syscall (write) | 1 | 9 | outputs to fd 9 |
|-----------------|---|---|-----------------|

|                |    |   |      |
|----------------|----|---|------|
| syscall (exit) | 60 | - | done |
|----------------|----|---|------|

**## Where's the Byte?**

Need to find exact file offset:

```
```bash
```

```
xxd -s 0x3060 -l 16 bin/challenge.bin
```

```
'''
```

```
'''
```

```
00003060: eb3f 5880 771f 4148 31c0 0402 4831 f60f.  
?X.w.AH1...H1..
```

```
'''
```

See that 58?

That's our ``pop %rax``.

It's at offset 0x3062 in the file (two bytes after 0x3060).

I double-checked: .data section starts at file offset 0x3040, shellcode is at 0x3060, bug is at shellcode+2 = 0x3062.

Patching It

Quick python one-liner to flip the byte:

```
```bash
```

```
python3 -c "
```

```
with open('bin/challenge.bin', 'rb') as f:
```

```
data = bytearray(f.read())
```

```
print(f'Original: 0x{data[0x3062]:02x}')
```

```
data[0x3062] = 0x5f
```

```
print(f'Patched: 0x{data[0x3062]:02x}')
```

```
with open('bin/challenge_patched.bin', 'wb') as f:
```

```
f.write(data)
```

```
"
```

```
'''
```

```
'''
```

Original: 0x58

Patched: 0x5f

```
'''
```

## Sanity Check

Did I only change one byte?

Let's verify:

```
```bash
```

```
cmp -l bin/challenge.bin bin/challenge_patched.bin
```

```
'''
```

```
'''
```

12387 130 137

```
'''
```

Just one byte at position 12387 (that's $0x3062 + 1$ because `cmp` is 1-indexed).
Changed from octal 130 (0x58) to octal 137 (0x5f).

```
```bash
```

```
xxd -s 0x3060 -l 16 bin/challenge_patched.bin
```

```
'''
```

```
'''
```

00003060: eb3f 5f80 771f 4148 31c0 0402 4831 f60f.  
?\_w.AH1...H1..

```
'''
```

The 'X' (0x58) became '\_' (0x5f).  
`pop %rax` is now `pop %rdi`.

## Testing

First I need to make the directory and file that the shellcode wants to read:

```
```bash
```

```
mkdir -p directory/newdirectory
```

```
echo "shellcode worked!
```

```
- wayner" > directory/newdirectory/file.txt
```

```
```
```

The shellcode writes to fd 9 (weird choice but ok), so I need to redirect that to see the output:

```
```bash
```

```
./bin/challenge.bin 9>&1
```

```
```
```

```
```
```

Segmentation fault (core dumped)

```
```
```

Still crashes.

Expected since that's the original.

Now the patched one:

```
```bash
```

```
./bin/challenge_patched.bin 9>&1
```

```
```
```

```
```
```

```
shellcode worked!
```

```
- wayner
```

```
```
```

It works.

The shellcode opens the file, reads it, and writes to fd 9.

## ## TL;DR

| What | Value |
|------|-------|
|------|-------|

|             |        |
|-------------|--------|
| File offset | 0x3062 |
|-------------|--------|

|          |                 |
|----------|-----------------|
| Original | 0x58 (pop %rax) |
|----------|-----------------|

| Patched | 0x5F (pop %rdi) |

|      |        |
|------|--------|
| Size | 1 byte |
|------|--------|

## ## Why fd 9?

This threw me off at first.

The shellcode writes to fd 9 instead of stdout (fd 1).

I'm guessing this is for pwn.college's grading system where they capture output from specific file descriptors.

You need `'9>&1'` to actually see anything.

## ## Misc Notes

**\*\*The jmp-call-pop trick:\*\*** Classic way to get a data address in shellcode without hardcoding it.

Jump forward, call backward (which pushes the return address), pop it into a register.

## \*\*Why XOR the 'A'?

**\*\* Shellcode can't have null bytes or it breaks strcpy and similar functions.**

So the string ends with 'A' (0x41) and gets XORed with 0x41 to make a proper null terminator.

\*

## FAQs

---

### What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 10 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

### When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

### Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

### What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

### I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.